

openvaf-r — Full Change Report

Every modification and its reason (original → current)

Ngspice + OpenVAF Enhancements project

July 2026

Contents

openvaf-r — Full Change Report	3
1. Lexer — openvaf/lexer/ (lib.rs, cursor.rs)	3
2. Tokens and grammar — openvaf/tokens/, openvaf/parser/, veriloga.ungram, sourcegen/	3
3. Preprocessor — openvaf/preprocessor/ (+ basedb diagnostics)	4
4. Syntax / AST — openvaf/syntax/ (ast.rs, generated/nodes.rs, expr_ext.rs, node_ext.rs, name.rs, validation.rs, error.rs)	4
5. Elaboration — openvaf/hir/src/elaborate.rs (new file)	5
6. Definition layer — openvaf/hir_def/ (+ openvaf/hir/ mirrors)	6
7. Types and validation — openvaf/hir_ty/	6
8. Lowering — openvaf/hir_lower/	7
9. MIR core and optimization — openvaf/mir*	8
10. DAE construction — openvaf/sim_back/	9
11. OSDI code generation — openvaf/osdi/	9
12. Driver, libraries, and infrastructure	10
13. The test suite — openvaf/openvaf/tests/, lib/mini_harness/, test_data/, integration_tests/	10
14. Per-enhancement index (compiler-touching enhancements)	10

openvaf-r — Full Change Report

Every modification applied to the OpenVAF-Reloaded compiler in this project, with its reason. The baseline is the pristine OpenVAF-Reloaded source tree (as vendored in the project's `original/` snapshot); the current state is the tree committed in this repository under `OpenVAF-master-20260610/`. Each entry links the enhancement write-up in `enhancements_doc/` that carries the full engineering detail and the verifying example suite. The companion document `ngspice_changes_full-report.md` covers the simulator side.

Maintenance note: this report is updated whenever an enhancement touches compiler sources. The per-enhancement index at the end tells you the last change it covers.

Scope summary. ~120 modified source files across the compiler pipeline, two new files (`hir/src/elaborate.rs` — the elaboration pass — and `hir_def/src/item_tree/diagnostics.rs`), plus three classes handled wholesale: the `test_data/` fixtures and snapshots (95 files — new test inputs and snapshots regenerated whenever a feature legitimately changed descriptor contents), the `integration_tests/*/*.osdi` reference objects (regenerated at each ABI bump), and the removal of the AGPL `va-cask` submodule. The compiler builds warning-free; every change below was regression-gated by the example-suite arsenal, the integration suite, and the `VA_TEST` industry-model corpus.

1. Lexer — `openvaf/lexer/` (`lib.rs`, `cursor.rs`)

- Based integer literals (`8'hFF`, `'sb101`): the lexer had only a commented sketch; underscores crashed it. Both the sized and unsized forms now lex, requiring a digit after the base (a silent-0 trap otherwise) ([E-46](#)).
- Don't-care digits `x/X/z/Z/?` accepted in the binary/octal/hex digit runs for `casex/casez` items ([E-78](#)).
- Escaped identifiers (`\my-net!`) lexed per the LRM ([E-46](#)).
- `// comment at EOF` hang fixed: with no trailing newline the line-comment loop spun forever on the EOF character — the only EOF-unsafe loop in the lexer ([E-35](#)).

2. Tokens and grammar — `openvaf/tokens/`, `openvaf/parser/`, `veriloga.ungram`, `sourcegen/`

`tokens/src/parser/generated.rs` gains the keywords and node kinds each language feature needed, threaded through `parser/src/grammar/*` (`stmts.rs`, `expressions.rs`, `items.rs`, `items/module.rs`), the grammar source of truth `veriloga.ungram`, and the `sourcegen/` generators (`ast/src.rs`, `hir_builtins.rs`, `mir_instructions.rs` — the templates the generated token/AST/builtin/instruction tables come from):

- `do ... while` post-test loops ([E-19](#));
- `paramset/endparamset` blocks ([E-21](#));
- `{...}` concatenation and `{n{...}}` replication as real operators, with `'{...}` remaining the typed aggregate ([E-34](#));
- array-literal expressions actually parsed (they were fully scaffolded but unreachable) ([E-4](#), [E-14](#));
- vectored (bus) net declarations and bit-selects ([E-3](#)), multi-index bit-selects for N-D arrays ([E-15](#)), and the LRM name-then-range declaration order ([E-18](#));
- module instantiation items ([E-5](#)), `generate for/genvar` ([E-8](#)), and `generate if/generate case` with optional block labels (previously mis-parsed into a broken `GENERATE_FOR`; [E-67](#));
- `defparam` — a reserved word with no grammar rule ([E-58](#));
- event OR lists `@(cross(...)` or `timer(...))` via a new or keyword and looped event grammar ([E-59](#));
- `casex/casez` sharing the case rule, the keyword token carrying the flavor ([E-78](#));

- `@(initial_step)/@(final_step)` with analysis-phase lists (E-7, E-53);
- indirect branch assignment `V(x): V(y) == expr;` (E-2);
- the `<<</>>>` arithmetic-shift tokens (plus a pre-existing lexer bug fix) (E-6);
- hierarchical parameter override targets (`#(.blk.p(4))`): the extra `.segments` are consumed so the parse stays clean instead of cascading -- elaboration then rejects the block-scoped override with a targeted diagnostic (E-87);
- hierarchical branch reference tails (`.branch(a,b) / .branch(<p>)`) swallowed into the path node so the enclosing item's CST stays whole for the elaboration rewrite (a keyword in path position used to shred the item, leaving the hole scanner truncated text) (E-86);
- part-selects in instance connections (`inst (out[3:2], in)`): the bit-select bracket accepts an optional `: expr`, with the colon token in the CST distinguishing a range from multi-dimensional indexing — no new node kind (E-85);
- module-level **generate for/if/case** without the optional **generate/endgenerate** keywords: `module_items` handled a `generate ... endgenerate` region but had no bare `FOR_KW/IF_KW/CASE_KW` arm, so a keyword-less loop at module scope fell to error recovery — unexpected token 'for', or a silent drop of the loop when a following analog block let recovery resync. New arms parse it into the same `GENERATE_FOR/IF/CASE` nodes the nested/generate-wrapped forms produce (E-96);
- panic-free direction parsing: `port_decl/func_arg` asserted the next token was a direction (`bump_ts`), which any non-Verilog text reaching a module-head port list turned into a compiler crash; both now emit a diagnostic with one token of forced progress (E-84);
- operator precedence corrected against LRM Table 4-2: `%` bound tighter than `*/` (so `6*7%4` evaluated to 18, not 2), and `xnor` split from `xor` (E-38);
- derived-nature parents emitted the wrong node kind (`NAME_REF` where the AST wanted `Path`), leaving the entire inheritance machinery unreachable (E-39).

3. Preprocessor — **openvaf/preprocessor/** (+ **basedb** diagnostics)

- Ten compiler directives that previously hard-failed as undefined macros: ``default_discipline`, ``celldefine/`endcelldefine`, ``unconnected_drive/`nounconnected_drive`, ``timescale`, ``line`, ``pragma`, ``undefineall`, ``default_nettype` (E-6).
- ``default_transition` parsed and threaded to `transition()` lowering (E-47).
- Recursive macro expansion crashed the compiler (stack overflow, direct and mutual): the `MacroRecursion` diagnostic existed but was never emitted, and its report renderer was a `todo!()`. The guard pushes around the macro *body* expansion only — an entry-scoped guard false-positives on the legal `QUAD(x) = TWICE(TWICE(x))` nesting (E-65).

4. Syntax / AST — **openvaf/syntax/** (**ast.rs**, **generated/nodes.rs**, **expr_ext.rs**, **node_ext.rs**, **name.rs**, **validation.rs**, **error.rs**)

- Typed AST nodes for every new construct above (generated from the ungrammar).
- Literal decoding (`expr_ext.rs`): based-int parsing with size/sign handling (E-46) and the mask-aware variant returning (`value`, `x_mask`, `z_mask`) for `casex/casez` (E-78); a compiler crash on large bare-integer-shaped literals fixed by falling back to a float constant (E-4).
- String unescaping rewritten as a single pass: the sequential `str::replace` chain corrupted overlapping escapes (`\\n`) and octal `\\ddd` was missing; all consumers route through one function (E-48).
- **Name::resolve** ate the last character of escaped identifiers (the committed `std.va` snapshot had baked the bug in as `logi`) (E-46).

5. Elaboration — `openvaf/hir/src/elaborate.rs` (new file)

The compile-time flattening pass: every stage downstream of the parser is architected around exactly one flat module per artifact, so hierarchy is resolved by recursively inlining instantiated modules — alpha-renamed per instance, ports bound to the caller's nets, parameters bound to overrides — into an ordinary flat module *before* the rest of the pipeline runs. Built for module instantiation (E-5) and grown into the home of every structural feature:

- generate `for/genvar` unrolling (E-8), nested loops, anonymous blocks, generate `if/case` with elaboration- time constant folding, and the `genvar`-substitution fix (genvars in ordinary expressions were routed through identifier renaming, which re-escaped `1e3*(i+1)` into a broken escaped identifier; they now become literal-value holes) (E-67);
- implicit nets from undeclared instance connections, discipline derived from the connected port, prefixed per instance so no cross-instance shorts (E-41);
- hierarchical names `u1.u2.node` and `$root` via an instance-chain map and a hole scanner (E-49);
- **defparam** resolved through the same chain rewrite, with the LRM-mandated precedence over instance `#( )` overrides (E-58);
- net concatenation in port connections expanded bit-by-bit onto vectored ports (E-59);
- bus slicing onto instance arrays and bus ports (E-5); paramset twin-module rendering (E-21); net initializers preserved through re-rendering (E-45); escaped identifiers re-rendered correctly (E-46);
- undefined instantiation targets are a hard error — they used to be silently dropped from the rendered output, turning a typo'd module name into an invisible open circuit; discipline-named misparses (`electrical out[0:2]`; parses as an instantiation) and paramsets dropped over unresolvable targets get tailored messages (E-84);
- name-then-range net/port declarations (`input in[0:2], electrical out[0:2]`, LRM 3.6/3.7): a textual pre-pass rewrites the `<head> <name>[range]` form to the range-then-name form (Enhancement-3), reusing all bus/port machinery; the `(-after-range` instance rule keeps instance arrays untouched (E-89). Extended to multi-name lists (`input a[0:1], b[0:3], c;`), split into one range-then-name declaration per name with per-name widths; a multi-dimensional name is left untouched (unsupported in both orders) (E-91);
- parameter-dependent declaration widths (`electrical [0:N-1] out;; real w[0:N-1];`): a textual pre-pass folds a declaration range whose bounds reference a parameter to a literal range, using the module's constant-integer parameter defaults (a fixpoint resolves one default through another; `from/exclude` constraints are skipped). Only declaration ranges (with a `:`) are folded, not bit-selects; the shared constant-integer evaluator gained an optional parameter map (empty map = the E-88 literal-only behavior). The width is fixed at the default -- a structural parameter, since OSDI has one node count per module (E-67/88 decision) (E-91). A parameter that shaped a width is then frozen to a **localparam** (`freeze_width_parameters`): a multi-parameter declaration is split so only the structural names freeze, and a range constraint is dropped from a frozen name. This keeps structure and behaviour consistent under a netlist override -- without it, overriding a width parameter that also bounds a runtime loop left the frozen array size behind, a silent out-of-bounds (E-92);
- the legacy **generate** `<id> (start, end [, incr])` statement (obsolete Verilog-A 1.0 analog-block loop-unroll, LRM Annex C.4) unrolled by a textual pre-pass: the index substitutes to a literal per iteration (with bit-select bracket folding, since a bus bit-select needs a literal index), constant bounds only -- a parameter bound cannot shape the unroll (E-67 scope decision) (E-88);
- ``__FILE__`/``__LINE__` expanded by a textual pre-pass run before the other passes (preprocessor tokens are (kind, span) pairs into existing source and cannot carry synthesized literals): ``__FILE__` becomes the root file's basename (machine-portable provenance, the E-58 rule), ``__LINE__` the exact 1-based line; string/comment occurrences are skipped and replacements are inline so later line numbers stay true (E-85);
- part-select actuals sliced onto bus ports in `bind_port (base[msb:lsb]`, constant bounds; positional and named forms; width-1 slices onto scalar ports degrade to the bit-select) with the same

ascending bit-order convention as full-bus slicing (E-85);

- block-scoped parameter override rejected with a targeted message (a named instance override `#(.blk.p(4))` naming a hierarchical target is collected and bailed like the unknown-module errors; block-scoped parameters are local to their block and take their value from their initializer, which may reference the module's parameters) (E-87);
- hierarchical branch probes (E-86): the top module's absolute chain map rides into every inlined child so SIBLING bodies resolve `V(top.a1.b)/$root...` references; unnamed-branch forms expand to the flattened node pair; port-branch probes `I(inst.branch(<p>))` read a 0V ammeter synthesized at flattening (which also fixes child-declared branch `(<p>)` port branches, broken after inlining); hierarchy-bound monitor modules are omitted from standalone output; ground/net-type declarations in inlined children keep their keyword;
- **\$port_connected** resolved at flattening time to a literal `(1)/(0)` per instance — after inlining, an open port is just a synthesized local net, so the builtin used to fail validation in exactly the unconnected case it exists to detect; top-level modules keep the native OSDI connected-flag path (E-84).

6. Definition layer — **openvaf/hir_def/** (+ **openvaf/hir/** mirrors)

`body.rs/body/lower.rs/body/pretty.rs`, `expr.rs`, `item_tree*`, `nameres*`, `builtin.rs`, `data.rs`, `types.rs`, `db.rs`, plus the new `item_tree/diagnostics.rs` and the hir façade (`body.rs`, `lib.rs`, `db.rs`, `diagnostics.rs`):

- statement/expression collection for every new construct: `do...while` (E-19), `repeat` and `disable` (E-9), event OR lists (E-59), `concat/` replication (E-34), array aggregates and whole-array assignment (E-14), `CaseKind` + per-item don't-care masks with a stray-literal list (E-78);
- N-D array machinery: per-dimension size lists, multi-index bit-selects, per-element parameter expansion (E-15); declaration initializers on (N-D) arrays via a `Var::array_index` leaf split (E-43);
- **localparam** made genuinely non-overridable (it behaved exactly like parameter) with derived localparams still tracking inputs (E-9); paramset lowering as twin modules whose bound params are localparams with override expressions (E-21);
- name resolution: electrical ground `gnd`; ordering (E-9), nature-attribute access (`net.potential.abstol` — the third "scaffolded-but-unwired resolver boundary" found) (E-45), derived-nature resolution (E-39);
- **builtin.rs** (the system-function registry): the noise-table pair (E-9), the full `$random/$dist_*/$rdist_*` family (E-10), file I/O and string functions (E-11), the last fallback group (`$simprobe`, aliases, `plusargs`) — after which `is_unsupported()` is empty (E-12), `$simparam$str` (E-25), `$realtime` (E-59), `$table_model` (E-16);
- new diagnostics file: wrong-arity calls produced invalid HIR and crashed downstream — now a proper diagnostic (functions and parameters both) (E-43);
- multiple analog blocks collect into `entry_stmts` in source order — the as-if-concatenated LRM semantics, by construction (E-60);
- bus-port node ordering (`item_tree/lower.rs`): a non-ANSI header bus port (`module m(in, y); input [0:2] in;`) had its first bit merged into the header placeholder but the remaining bits appended after later ports, scrambling OSDI terminal order whenever the bus was not the last port. A pre-scan of body port widths now expands the bus into contiguous bits in header order at placeholder-creation time (E-90).

7. Types and validation — **openvaf/hir_ty/**

`inference.rs` (+ `inference/fmt_parser.rs`), `builtin.rs` + `builtin/generated.rs` (signature tables), `lower.rs`, `types.rs`, `validation.rs`, `validation/body.rs`, `diagnostics.rs`:

- signature tables — a recurring defect source: ANALYSIS made variadic (E-30); `$table_model`

given shape-synthesized varargs signatures for any dimension (a generic-varargs fallthrough *truncated* multi-arity signature lists — varargs builtins with signature lists need their own inference arm) (E-40); TRANSITION's table was one argument short (3-argument calls crashed, 4-argument worked by accident) (E-47); `transition()` input typed Real, a `$dist` typo fixed (E-49); SIMPARAM_STR returned Real instead of String (E-25);

- format-string inference (`fmt_parser.rs`): terminates on every conversion character and reports it, so `%5d/%-8s/%08x` type their arguments correctly — previously only real conversions accepted flags/width (E-71);
- array inference: element-wise array case, array-literal function arguments (which previously bound nothing and silently returned 0), integer arrays typed Real, output-literal args accepted silently — all fixed (E-33); whole-array function arguments/outputs/returns (E-18, E-20, E-23); untyped function args default to Real instead of an ICE masquerading as an initializer bug (E-43);
- **Type: :base_type** infinite loop on any array-type check (it looped on `self` instead of the cursor) — hung the compiler on simple type mismatches (E-14);
- string handling: ternary over strings (SELECT + string binary ops appended to the operator tables) (E-37);
- validation: loop bodies get their own context so the analog-operator restriction reports "loops" (LRM 4.5.1), not "conditions" (E-70); call-graph cycles among analog functions are clean errors naming the cycle instead of a recursive-inliner stack overflow (E-59); `casex/casez` restrictions (stray don't-care literals, x digits under `casez`, non-integer discriminants) (E-78); domain discrete with natures rejected per LRM 3.6.2.2 (E-50); parameter defaults exempt from their own ranges (the CMC "feature disabled" idiom — stock rejected `diode_cmc`, BSIM-CMG, PSP-HV and the HiSIM family at setup) while given values stay fully validated (E-56);
- part-selects outside port connections diagnosed (the E-78 stray-list pattern: body lowering collects them, validation reports a dedicated error naming the supported connection form) (E-85);
- contributions to port branches diagnosed (`ContributeToPortFlow`, with the declaration site labeled): `I(pb) <+ ...` on a branch (`<p>`) `pb`; slipped through the write path unvalidated and panicked during lowering (E-84);
- contributions to an all-**ground** branch diagnosed (`ContributeToGround`): `V(gnd) <+ ... / V(gnd, gnd) <+ ...` reduce both endpoints to the fixed 0 reference (no unknown) and hit an `unreachable!()` in `lower_contribute_unnamed_branch` — an ICE. The write path now checks the branch's `is_gnd` nodes and reports a clean error; a real node-to-ground branch and a `V(gnd)` probe are untouched (E-97).

8. Lowering — `openvaf/hir_lower/`

`expr.rs`, `stmt.rs`, `ctx.rs`, `callbacks.rs`, `parameters.rs`, `state.rs`, `fmt.rs`, `lib.rs`:

- analog operators: `laplace_nd/np/zd/zp` via exact state-space realization (E-4) with complex pole/zero pairs per the LRM's (re, im) convention (E-31); `zi_*` via bilinear transform (E-6); `slew/transition` as a saturating tracking loop (E-6) with the LRM negative `max_neg_rate` convention honored (the sign defect made `slew` ignore its input and ramp away) (E-61) and a DC-singularity clamp via an `EnableIntegration` identity (E-47); `idtmod`'s modulo wrap moved off the reactive residual (it diverged) and an argument- index bug fixed (E-27); `idt` initial conditions survive into transient (E-28); the `idt` assert/reset forms with smooth reset dynamics (E-52); `limexp` kept stateless as a documented decision (E-13);
- **\$table_model**: 1-D piecewise-linear lowered to differentiable MIR so autodiff yields the Jacobian for free (E-16); N-D multilinear as recursive 1-D (E-17, E-40); natural cubic splines with the precomputed moment matrix (E-22);
- events: `cross/above/timer` with persistent previous-value slots (E-8); OR lists folded with a select-based bool-or (a raw `ior` ICEs const-eval) (E-59); final-step gating and phase-list AND-ing (E-53);
- variable persistence: genuine per-instance `hidden_state` storage behind a two-pass MIR build

(E-7), typed slots so integer state stopped crashing instruction selection (E-32);

- callbacks (callbacks.rs, fmt.rs): the `$display` family with the full `[flags][width][.precision]` surface and `%h/%b/%r` translation (E-71); file I/O and string formatting through a generalized `PrintDst` sink (E-11); the deterministic seed-and-salt RNG lowering (E-10); `$simplparam$str` (E-25); simulation-control return flags (E-55); `$discontinuity` (E-24);
- operator fixes: unary `~` emitted arithmetic negate; the constant folder sign-extended `>>` where the runtime was unsigned — `const` and runtime disagreed (E-37);
- whole-array coercion (expr.rs, stmt.rs): inference records a coercion of a whole array as a cast on the array *expression*, but `lower_array_elems_impl` decomposes the array and lowers each element itself, so `lower_expr's needs_cast()` never saw it — the cast was silently dead, and every new array-consuming context re-inherited the same crash (an integer element reaching a float MIR op, which `const-eval` has no case for: *"invalid operation fdiv Int(1) Float(..)"*). Patched per-call-site four times — the case discriminant (E-33), `laplace_*/zi_*` integer-literal then integer array-variable coefficients, and an integer case item against a real discriminant — before being fixed at the chokepoint, which now honours the recorded cast for every consumer (E-214). The explicit `coerce_real` flag remains for consumers whose element type is fixed by the language rather than by an inferred cast (a coefficient vector is real per LRM 9.19, and inference records no cast for it);
- masked `casex/casez` comparisons (two `iands` ahead of the existing integer equality) (E-78); array function-argument writeback (E-20) and array returns via a function-named var-array (E-23);
- named port branches (branch (<p>) pb; — LRM 3.7.2): a flow probe of a `PortFlow`-kind branch routes through the same `CurrentKind::Port` param as a direct `I(<p>)`, so E-29's defining equation covers both spellings; probing one used to hit `unreachable!()` (E-84);
- literal `if` conditions lower only the taken branch — a dead analog operator (`transition()` under `if ((0))`), the exact shape the `$port_connected` rewrite produces) survived `const-folding` as a detached-but-interned op whose state setup read optimized-away values and aborted codegen (E-84).

9. MIR core and optimization — **openvaf/mir***

mir/ (instructions.rs + instructions/generated.rs, dfg.rs with dfg/phis.rs and dfg/uses.rs, dominators.rs, flowgraph.rs with flowgraph/transversal.rs, layout.rs, cursor.rs, lib.rs, builder/generated.rs), mir_opt/ (simplify_cfg.rs, simplify.rs, const_eval.rs, global_value_numbering.rs), mir_autodiff/ (builder.rs, lib.rs, live_derivatives.rs), mir_llvm/ (builder.rs, intrinsics.rs), mir_build/, mir_interpret/:

- three pre-existing general CFG bugs, all found via event-counter verification: a dangling-reference bug in unreachable-block removal, a multi-exit post-dominance bug in the dominator-tree builder, and a block-merge bug that could corrupt which block the DAE builder treated as the exit (E-8);
- the post-dominator sink-root pathology that let dead-code elimination delete op-dependent `$fatal` calls (side-effecting callbacks under op-dependent control now stay in eval; control dependence computed by arm-reachability) (E-55);
- `split_tainted.rs` tolerates layout-detached branch instructions (a branch whose condition `const-folded` has no CFG effect to taint; it used to unwrap and panic) (E-84);
- `const-eval` agreement with runtime semantics for shifts and the bitwise-not fix (E-37);
- `autodiff`: noise operators process before any `ddt` (a shared `ddt` evaluated between two noise operators silently dropped the second source), and the reactive coupling of a `ddt`-shaped noise wave is registered so small-signal pruning keeps it (E-54);
- **llvm.fabs.f64** registered in the LLVM intrinsics table — it never had been, and the signed noise-power convention needed it (E-42);
- new opcodes/instructions supporting the operator work (barrier/ integration-identity forms; MIR deliberately has no `fabs`, so lowering composes it) (E-47, E-52).

10. DAE construction — **openvaf/sim_back/**

dae.rs + dae/builder.rs, topology* (incl. linearize.rs, small_signal_network.rs), noise.rs, context.rs, lib.rs:

- implicit equations for indirect branch assignment (one new unknown
 - residual per statement (E-2) and the absdelay synthetic two-node form (E-1) — with equation indices kept stable by mapping dead/collapsed unknowns to zero-contribution placeholders instead of renumbering;
- port-flow probes **I(<port>)**: a TODO stub returning 0 became a synthesized DAE unknown mirroring the node's KCL residual (E-29);
- probe-only branches: probing a never-contributed branch read 0 and an open circuit — a 0 V-source pass materializes them, enabling ideal ammeters and flow-only signal-flow disciplines (E-36);
- noise factors (noise.rs): equation-path noise was silently lost (never attached to the DAE); factors generalize to $re + j\omega \cdot im$ with no extra matrix unknowns (E-54); same-named source grouping metadata (E-42);
- idt reset-mode charge dynamics and conditional step bounding (E-52).
- voltage-source branches feeding internal nodes were open circuits at DC: the small-signal (noise/ac_stim) pruner keyed node registration on the branch's LIVE voltage unknown, so a pure $V(a, f) \leftarrow \text{expr}(V(a, f) \text{ never read})$ registered nothing and the node's conduction silently moved to the AC-only residual; any voltage-capable branch now disqualifies its nodes (E-86);
- probed **V(x, y) <- 0** branches were node-collapsed away, making **I(branch)** read zero; hint pairs whose branch current is a DAE unknown are suppressed (the unprobed collapse idiom is untouched) and `NodeCollapse::hint` tolerates suppressed pairs; pinned by the permanent `vsrc_internal_node` snapshot test. Behavior change: a collapsible branch whose current the model references (MVSG_CMC's access resistances carry noise on `flow(d, drc)`) stays a real 0V source — electrically identical, two extra matrix rows (E-86).

11. OSDI code generation — **openvaf/osdi/**

lib.rs, metadata.rs + metadata/osdi_0_4.rs, inst_data.rs, eval.rs, load.rs, setup.rs, compilation_unit.rs, ndatable.rs, stdlib.c, build.rs, reference headers:

- descriptor emission for every ABI evolution (see the ngspice report's ABI table): the absdelay/last_crossing info arrays (E-1, E-6), `OsdiNode.nodeset` (E-45), the `ac_stim` source array + `load_ac_stim` (E-51), stride-2 signed noise pairs in `load_noise` (E-54) — the version tuple lives in `osdi/src/lib.rs`;
- **metadata.rs**: the `PARAM_FLAG_FIXED` parameter-descriptor flag (a free bit of the `flags` field, additive — no layout/ABI change) set on every `localparam`, so the simulator can warn when a netlist tries to set a non-settable parameter (a `localparam`, or a structural width parameter frozen by Enhancement-92) (E-93);
- **eval.rs**: final-step flag gating (E-53); simulation-control return flags (E-55); the `ac_stim` large-signal value (was an unreachable!() panic) (E-26);
- **inst_data.rs**: absdelay slot layout (E-1); hidden-state slots typed by the variable (`ty_f64` was hardcoded — integer state fed `f64` loads into integer MIR ops and aborted instruction selection) (E-32);
- **compilation_unit.rs** (print codegen): the `%b segfault` — the binary-formatted string was remembered for `free()` but never pushed to the `snprintf` argument list, so the matching `%s` consumed a garbage pointer (E-71); the print-callback machinery with its shadowed-fun and volatile-table IPO traps (E-11);

- **stdlib.c**: the file-descriptor table behind `$fopen/$fdisplay/...` (E-11); the `splitmix64 osdi_rng_*` runtime (E-10); a `simparam_str` loop/return bug (E-25);
- **ndatable.rs**: `ddt/idt` natures resolved through `resolve_nature_index` instead of panicking on derived natures (E-39);
- **setup.rs**: range validation with default-exemption (E-56); a 64-line abandoned unsafe experiment (`VoidAbortCallback`) deleted in the warning cleanup (E-66);
- **build.rs**: the copied-target `stdlib` compilation trap fixed (E-10).

12. Driver, libraries, and infrastructure

- `openvaf-driver/src/main.rs`: hard errors now exit non-zero — the error arm printed the failure but fell through to a success exit, so every elaboration failure looked like a successful compile to shell scripts (a quirk first noticed during E-58's work) (E-84);
- `openvaf-driver/src/crash_report.rs`, `linker/src/lib.rs`, `lib/bforest/` (`map.rs`, `pool.rs`, `set.rs`), `lib/typed_indexmap/` (`set.rs`): the zero-warning cleanup ($44 \rightarrow 0$ across 13 crates: lifetime annotations, dead code, nightly cfgs, an unused LLVM-prefix probe) (E-66);
- `basedb/` (`ast_id_map.rs`, `lints.rs`, `lib.rs`, `diagnostics/preprocessor_error.rs`, `diagnostics/syntax_error.rs`): AST-id plumbing for elaboration-created items (the `AstId::from_erased` placeholder used by array returns; E-23), `preprocessor/syntax` diagnostic rendering for the new errors;
- `.llvm-version` pins the LLVM 18 toolchain expectation; the `hir` and `preprocessor` `Cargo.tomls` carry the dependency additions their new code needed.

13. The test suite — **openvaf/openvaf/tests/**, **lib/mini_harness/**, **test_data/**, **integration_tests/**

The fork's own integration suite over real compact models had never run in this project: its OSDI loader was frozen at ABI 0.4, its optional VACASK legs panicked on a never-initialized submodule, and the tests hide behind `RUN_DEV_TESTS=1`. Fixed test-side only (E-68):

- `tests/load/osdi_0_4.rs` + `load/mod.rs`: loader structs synced to ABI 0.7; `mock_sim/mod.rs`: stride-2 noise convention; `mini_harness`: missing directories skip with a note instead of panicking;
- VACASK removed entirely (`.gitmodules`, the submodule, 34 harness legs and their stale snapshots): AGPL-3.0 cannot be vendored here;
- `test_data/` (95 files as a class): new fixtures for new features and snapshots regenerated whenever descriptor contents legitimately changed — every regeneration reviewed (the diffs are the project's own features: `flow(<port>)` unknowns, probe-only branches, implicit-equation nodes); `integration_tests/**/*.osdi` regenerated at each ABI bump.

14. Per-enhancement index (compiler-touching enhancements)

Enhancement	Pipeline areas	One line
E-1	<code>sim_back</code> , <code>osdi</code>	<code>absdelay()</code> synthetic-node DAE + descriptor arrays
E-2	<code>parser</code> → <code>hir_lower</code> , <code>sim_back</code>	indirect branch assignment (implicit equations)
E-3	<code>parser</code> , <code>hir_def</code>	vectored (bus) nets with bit-selects
E-4	<code>hir_lower</code> , <code>syntax</code> , <code>parser</code>	<code>laplace_*</code> state-space + array-literal parsing
E-5	<code>elaborate.rs</code> (new), <code>parser</code>	module instantiation via compile-time flattening

Enhancement	Pipeline areas	One line
E-6	preprocessor, parser, hir_lower, osdi	directives, shifts, slew/transition, zi_*, last_crossing
E-7	hir_lower (state), osdi	@(initial_step) gating + variable persistence
E-8	parser, elaborate, hir_lower, mir/mir_opt	generate for + events; three general CFG bug fixes
E-9	hir_def, hir_lower, sim_back	noise tables; localparam/ground/strings; repeat/disable
E-10	hir_lower, osdi stdlib	\$random/\$dist_* deterministic RNG
E-11	hir_lower, osdi	file I/O + string functions (PrintDst)
E-12	hir_def, hir_lower	last unsupported builtins as LRM fallbacks
E-13	hir_lower	limexp kept stateless (documented decision)
E-14	hir_def, hir_ty, hir_lower	array literals/params/dynamic indexing; base_type hang
E-15	hir_def, hir_ty, hir_lower	N-D arrays
E-16/17/22/40	hir_ty, hir_lower	\$table_model: 1-D, N-D multilinear, cubic spline, varargs sigs
E-18/20/23/33	hir_ty, hir_lower, basedb	arrays in analog functions: in/out/return/literals + array case
E-19	tokens→hir_lower	do ... while
E-21/44	parser, elaborate, hir_def, sim_back	paramset + hidden system parameters
E-24/55	hir_lower, mir, osdi	\$discontinuity; \$finish/\$stop/\$fatal honored
E-25	hir_ty, osdi stdlib	\$simparam\$str
E-26/51	osdi, sim_back	ac_stim: crash fix, then full AC-RHS injection
E-27/28/52	hir_lower, sim_back	the idt family: modulo, IC, assert/reset
E-29/36	sim_back	port-flow probes; probe-only branches
E-30	hir_ty, hir_lower	variadic analysis()
E-31	hir_lower	complex poles/zeros in root forms
E-32	osdi inst_data	integer persistent state (crash fix)
E-34	tokens→hir_lower	{...} concat + {n{...}} replication
E-35	lexer	EOF comment hang
E-37/38	hir_lower, mir_opt, parser	operator + precedence audits (~, >> fold, % level)
E-39	parser, hir_ty, osdi	derived natures unwired at the node-kind boundary
E-41/49/58/59	elaborate	implicit nets; hierarchical names; defparam; port concat + OR lists
E-42/54	sim_back noise, mir_autodiff, mir_llvm, osdi	correlated + node-free complex noise factors
E-43	hir_def (+diagnostics), hir_ty	initializers completed; arity diagnostics
E-45	osdi, hir_def, elaborate	nodesets + nature-attribute access (ABI 0.5)
E-46/48	lexer, syntax	escaped ids, based literals, single-pass unescaper
E-47	preprocessor, hir_ty, hir_lower	`default_transition + transition fixes
E-50	hir_ty validation	domain-binding validation
E-53	hir_lower, osdi eval	@(final_step) + phase lists
E-56	hir_ty, osdi setup	parameter defaults exempt from ranges

Enhancement	Pipeline areas	One line
E-59	tokens→hir_lower, hir_ty	\$realtime; OR lists; recursion diagnostics
E-61	hir_lower	slow sign fix (operator-args audit)
E-65	preprocessor	macro-recursion guard
E-66	13 crates	zero-warning cleanup (44 → 0)
E-67	tokens, parser, syntax, elaborate	generate audit: genvar fix, nesting, if/case
E-68	tests, mini_harness	integration suite enabled; VACASK removed
E-70	hir_ty validation	loop-context diagnostics
E-71	hir_ty fmt, hir_lower fmt, osdi	display-format surface + %b segfault
E-78	lexer→hir_lower, hir_ty	casex/casez don't-care masks
E-84	parser, elaborate, hir_ty, hir_lower, mir_opt, driver	LRM example sweep: 6 defect fixes (port-branch panic, parser robustness, silent undefined modules, \$port_connected on open ports, dead-op codegen, exit codes)
E-85	parser, elaborate, hir_def, hir_ty	`__FILE__`/`__LINE__` + connection part-selects (the last two sweep findings)
E-86	parser, elaborate, sim_back	hierarchical branch probes + 2 DAE fixes (V-source-to-internal open circuit, collapse-of-probed-branch)
E-87	parser, elaborate	block-scoped parameters (validated) + clean diagnostic for the illegal #(.blk.p()) override
E-88	elaborate	legacy generate <id> (start,end) analog-block loop-unroll (textual pre-pass)
E-89	elaborate	name-then-range net/port decls (textual normalize) + Annex E SPICE-primitives example library
E-90	hir_def (item_tree)	multi-bit input bus port bit reads: pre-scan body widths so a non-last bus port's bits stay contiguous in header/terminal order
E-91	elaborate	multi-name name-then-range declarations + parameter-dependent declaration widths (folded from the parameter default)
E-92	elaborate	freeze structural (width) parameters to localparam (split multi-parameter decls) so a netlist override cannot desync the frozen width from behaviour
E-93	osdi (metadata)	flag localparam OSDI parameters non-settable (PARAM_FLAG_FIXED, additive) so ngspice can warn on a netlist override (ngspice side too)
E-96	parser (module grammar)	parse a module-level generate for/if/case without the optional generate/endgenerate keywords
E-97	hir_ty (validation)	clean diagnostic (was an ICE) for a contribution to an all-ground branch (V(gnd) <+ ...)
E-101	hir_ty (builtin), mir_opt / mir_interpret / mir_llvm	\$clog2: accept one argument (INT_MATH_1, was a 2-arg signature) and compute ceil(log2 n) = bit_width(n-1) in all three backends (was floor(log2 n)+1, wrong on exact powers of two)

Enhancement	Pipeline areas	One line
E-102	parser (items), syntax (ungrammar + ast), hir_def (item_tree)	name-then-range array-valued parameters (parameter real c[0:2]); parameter() accepts post-name [range], lower_param resolves dims per name
E-103	mir_llvm (intrinsic)	register the llvm.ceil.f64 intrinsic (was missing; ceil() of a non-constant argument crashed codegen -- floor was registered)
E-104	syntax (name), hir_def (builtin), hir_ty (builtin), hir_lower (expr)	add \$rtoi (real->int, truncate toward zero via ficast((x<0)?ceil:floor)) and \$itor (int->real) conversion builtins
E-105	hir_lower (expr, callbacks), osdi (compilation_unit, stdlib.c)	\$sscanf/\$fscanf honour the format conversion base -- parse the format string, dispatch integer fields to base-specific scanners (%h/%x hex, %o octal, %b binary)
E-106	hir_ty (types, inference), hir (signatures), hir_lower (expr, callbacks), osdi (compilation_unit, stdlib.c)	string relational comparison (</<=/>/>=) via a lexicographic osdi_strcmp callback (RELATIONAL_COMPARISON adds the STR signature; a<op>b lowers to strcmp(a, b)<op>0)
E-107	syntax (name), hir_def (builtin), hir_ty (builtin), hir_lower (expr, callbacks), osdi (stdlib.c)	add \$fgetc(fd) single-character read as a FileOp::Getc -> osdi_fgetc (completes the file I/O family)
E-108	syntax (name), hir_def (builtin), hir_ty (builtin), hir_lower (expr, callbacks), osdi (stdlib.c)	add \$ungetc(c, fd) one-character pushback as a 2-arg FileOp::Ungetc -> osdi_ungetc (companion to \$fgetc)
E-109	hir_lower (callbacks), osdi (load)	correct noise_table (piecewise-linear in f) and noise_table_log (log-log, $P=10^{\text{lerp}(\log_{10} p, \log_{10} f)}$) interpolation to LRM 4.6.4.3/4.6.4.4 -- both were nonconformant (lin-log, and a mis-keyed log-freq input)

Enhancement	Pipeline areas	One line
E-147	hir_ty (validation/body)	fix exponential-time compilation of nested <code>?:</code>. Found by a robustness campaign (117 production CMC models + ~50 adversarial inputs + 4000 mutation-fuzz iterations). The body validator's <code>validate_expr</code> ends by recursing into children via <code>walk_child_exprs</code>; arms that validate their own operands return first (Call, Path), but the Select (ternary) arm did NOT -- it validated <code>cond/then_val/else_val</code> via <code>validate_condition</code> and then fell through, validating <code>then_val/else_val</code> a SECOND time, so a chain of N nested <code>?:</code> was validated 2^N times. Depth ~30 (reachable via macros) hung the compiler; sample-profiling confirmed an unbounded <code>validate_expr</code>↔<code>validate_condition</code> recursion with doubling call count. Fix: add <code>return;</code> to the Select arm (the operands are already fully validated). $O(2^N) \rightarrow O(N)$: depth 40 hang→0.09 s, depth 2000→1.6 s. Behaviour-preserving: all 117 models give the IDENTICAL verdict before/after (0 flips, head-to-head), BSIM4 (12.6k lines) still ~2.3 s, and <code>hir_ty/hir/hir_lower/sim_back</code> tests pass with no snapshot changes. Campaign also confirmed 0 panics/segfaults across 4000 fuzz iterations; remaining lower-severity findings (parser stack overflow on ~4k-32k-deep nesting, include self-recursion, huge array dimension) are documented follow-ups. Verified 7/7 (nested_cond_examples: depth 20/40/80/160 all <0.2 s, linear growth, correct piecewise value in ngspice)

Enhancement	Pipeline areas	One line
E-148	parser (parser, grammar/expressions), preprocessor (diagnostics, processor), basedb (diagnostics/preprocessor_error), hir_def (item_tree, item_tree/diagnostics, item_tree/lower), hir (elaborate)	compiler hardening: closes the three lower-severity robustness findings from E-147 (pathological input → crash/hang) by turning each into a clean bounded diagnostic. (1) Parser expression-depth limit: a shared <code>Parser::expr_depth</code> counter, incremented on each <code>atom_expr</code> (recursion) and per operator in the <code>expr_bp</code> loop (operator-chain / tree depth), bounds total expression-tree depth at 1000 and recovers to the next statement boundary when exceeded — so <code>----...x / x+1+1+... / ((...)) / sin(sin(...)) / 1?...:0</code> no longer overflow the recursive-descent parser OR a later recursive tree traversal (<code>expr_bp</code> split into a save/restore wrapper + <code>expr_bp_inner</code>; <code>atom_expr</code> into an <code>inc/check/dec</code> wrapper + <code>atom_expr_inner</code>; reuses the existing <code>err_recover/unexpected_tokens_msg</code> recovery). (2) <code>include</code> recursion cap: an <code>include_depth</code> counter on the preprocessor's <code>Processor</code> caps nesting at 64, emitting a new <code>PreprocessorDiagnostic::IncludeRecursionLimit</code> (rendered in <code>basedb</code>) instead of overflowing the stack on a self-including file — mirrors the E-65 macro-recursion guard. (3) Array element cap: a shared <code>array_elem_count</code> helper (product of <code> msb-lsb +1</code>, capped at $2^{20} \approx 1.05\text{M}$) + a new <code>ItemTreeDiagnostic::ArrayTooLarge</code>, applied at EVERY expansion site — variable arrays, parameter arrays, net/port buses, array function returns (all in <code>hir_def</code> item-tree <code>lower.rs</code>), and instance arrays in BOTH the item-tree lowering and the <code>hir</code> elaboration pass (which re-expands <code>dev s[0:N]()</code> into rendered text) — so <code>real x[0:100000000]</code> (and param/bus/instance equivalents) degrade to a single scalar with an error instead of exhausting memory. Behaviour-preserving: all 117 production CMC models give the IDENTICAL verdict before/after (0 flips, head-to-head), BSIM4 still ~2.3s; parser/preprocessor/hir_def/hir_ty/sim_back unit tests pass with no snapshot changes; 0 build warnings. Verified 17/17 (robustness_examples: 5 deep-expression shapes + self-include + 4 huge-array kinds all error in <1s no crash/hang with expected diagnostics; valid nested-ternary-30 / parens-100 / 100-term-sum / small arrays still compile). Fully resolves the E-147 campaign's remaining findings

Enhancement	Pipeline areas	One line
E-185	mir_autodiff (builder.rs)	autodiff audit: hypot & atan2 derivative fixes. A deep audit compared the AC small-signal conductance dI/dV (built by the mir_autodiff automatic differentiation that feeds AC/convergence/noise/pz) of a battery of nonlinear laws $I=f(V)$ against the ANALYTIC $f(V)$, and caught two builtins right in VALUE (DC correct) but wrong in DERIVATIVE -- the accidental-correctness pattern, first on the compiler side. (1) hypot(x,y): the autodiff rule computed $(x'+y')/(2hypot)$ -- <i>the sqrt(x) pattern $x'/(2sqrt)$ misapplied to a 2-arg function -- instead of the correct $(xx'+yy')/hypot$; 28% off at $V=0.7, y=0.5$, only accidentally correct at $x=0.5$. Fixed by splitting hypot from sqrt in inst_cache (cache holds hypot(x,y), not $2*hypot$) and a correct chain rule with the usual zero/one folding. (2) atan2(x,y): two bugs in the cached factors feeding the shared Pow Atan2 chain rule $(x'*c0 + y'*c1)*c2$ -- the common factor $c2$ was (x^2+y^2) where the rule MULTIPLIES, so it needed the reciprocal $1/(x^2+y^2)$; and $c1$ (the y' factor) was $+x$ where $d/du \text{ atan2} = (x'*y - y'*x)/(x^2+y^2)$ SUBTRACTS, so it needed $-x$. Wrong magnitude AND sign. Fixed: $c2 = 1/(x^2+y^2)$, $c1 = -x$. The fix is at the autodiff-rule level so it applies to resistive currents, reactive charges (verified via AC susceptance of ddt(hypot)), higher-order derivatives, and ddx.</i> verify_vafautodiff: 9 checks (both arg orders, sqrt-form equivalence, the $V=0.5$ accidental-correctness point, atan2 sign + atan2(V,V)->0 cancellation, reactive susceptance) + a 15-builtin regression battery confirming the rest were already correct. Regression 150/150.

Enhancement	Pipeline areas	One line
E-187	mir_opt (simplify.rs)	math-identity simplifier: invalid inverse-function cancellations. The algebraic simplifier (simplify_unary_op) rewrites $f(g(x)) \rightarrow x$ for function-inverse pairs. Six of these were applied unconditionally even though the cancellation is only valid when f is a true left inverse of g over ALL of g's range -- so they returned the raw inner x, WRONG for ordinary finite inputs, corrupting the DC value itself (a more severe class than the derivative-only autodiff bugs). (1) $\text{asin}(\sin(x)) \neq x$ for $x > \pi/2$ ($\text{asin}(\sin 3) = \pi - 3 = 0.1416$). (2) $\text{acos}(\cos(x)) \neq x$ outside $[0, \pi]$ ($\text{acos}(\cos 4) = 2\pi - 4$). (3) $\text{atan}(\tan(x)) \neq x$ for $x > \pi/2$ -- and this is a legitimate angle-WRAP idiom the optimizer silently defeated ($\text{atan}(\tan 2) = 2 - \pi$). (4) $\text{acosh}(\cosh(x)) \neq x$ for $x < 0$ ($\cosh$ even $\rightarrow x$). (5,6) $\text{sqrt}(xx)$ and $\text{sqrt}(x**2)$ are x, not x ($\text{sqrt}((-3)^2) = 3$, returned -3). The const-fold path (eval_unary) evaluates each op numerically and was always correct, so constant spot-checks passed -- the bug lived only in the symbolic cancellation on runtime values. Fix: Asin/Acos/Atan/Acosh decline to simplify (return None); the Sqrt arm returns None (MIR has no fabs, so the sqrt stays and computes x). Kept the cancellations valid over the whole real line ($\tan(\text{atan})$, $\ln(\exp)$, $\text{asinh}(\sinh)$, $\text{atanh}(\tanh)$, $\sinh(\text{asinh})$, $\cosh(\text{acosh})$, $\log_{10}(\text{pow}(10, \cdot))$). Rewrites unsound only for Inf/NaN ($x \cdot x > 0$, $x0 > 0$, $\sin(\text{asin}(x))$, $\exp(\ln(x))$) left as standard finite-math. Removed the now-unused F_TWO import. mir/mir_opt/mir_autodiff unit tests unchanged (13/7/19). New example mathident_examples/verify_mathident.py: 12 DC-value checks. Regression 151/151.

Enhancement	Pipeline areas	One line
E-186	mir_autodiff (builder.rs + lib.rs)	autodiff audit: real-modulo (%) derivative fix. The same AC-conductance referee (E-185) caught a third builtin right in VALUE, wrong in DERIVATIVE. Verilog-A real modulo lowers to the Frem opcode; $x \% c = x - \text{floor}(x/c)c$ is a slope-1 sawtooth in x, so $d/dx(x \% c) = 1$ away from the wrap points -- yet Frem was grouped with the genuinely-constant opcodes (floor, ceil, clog2, integer/bitwise ops, comparisons) and forced to (1) the live — derivative gate zero derivative() in lib.rs, which decides — with Frem listed, a modulo result was declared independent — return no — edge arm). Because the gate came first the AC/Jacobian a.dcsweep of $I = 1e - 3 * (V \text{ clog2/integer ops}$ correctly stay in the zero group. Applies everywhere the derivative is taken -- resistive I, reactive Q, ddx, higher orders. verify_vafautodiff extended to 14 checks (adds 5 modulo cases: two constant divisors, prefactor scaling, chain-rule $d/dV(V\%1)^2 = 2(V\%1)$, and floor/ceil staying 0), a separate 3-terminal probe confirming the variable-divisor branch, cross-checked via the ddx value path; the 19 mir_autodiff unit tests are unchanged and pass. Regression 150/150.

Enhancement	Pipeline areas	One line
E-213	syntax (lib.rs, parsing/tree_builder.rs), lexer (lib.rs), parser (grammar/paths.rs), preprocessor (parser.rs), examples/ vafcrash_examples/*	crash hardening: four compiler panics. Fuzzing openvaf-r with malformed Verilog-A found four distinct panics: instead of printing a diagnostic the compiler aborted with "OpenVAF encountered a problem and has crashed!" (exit 101) and directed the user to file a bug report -- for what is often just a typo. All are reachable from ordinary source mistakes; the headline case is a module merely missing its endmodule, one of the most common Verilog-A editing errors. BUG 1 -- EOF SPAN (tree_builder.rs): a parse error at end of file built its span as TextRange::at(self.text_pos,); text_pos is already the end of the source, so adding the previous token's length produced a span running PAST the end of the file -- 6..12 for the 6-byte input "module". Mapping it back to its file then failed assert!(range.end() <= self.range.end()) in FileSpan::with_subrange (preprocessor/sourcemap.rs), crashing the compiler WHILE TRYING TO PRINT THE ERROR ("subrange 6..12 -> 6..12 must fit into the total range 0..6"). FIX: an empty range at EOF -- exactly what the adjacent expected_at field already does. Also find_ctx_range (syntax/lib.rs) maps a position through half-open [start,end) ranges that never cover the EOF position itself (pos == last_range.end() compares Less against every range); it .expect()ed and panicked, now clamps. Triggers: missing endmodule; bare module; module header then EOF; unclosed analog begin; unterminated string. BUG 2 -- REAL LITERAL (lexer/lib.rs): e/E was consumed as an exponent marker without checking an exponent follows (eat_float_exponent()'s return, which reports whether a digit was seen, was discarded), so 1e became a Float token whose text does not parse as f64 and panicked in ast::StdRealNumber::value()'s src.parse().unwrap(). FIX: an e only joins the number when a digit (optionally signed) follows; a bare 1e lexes as 1 + identifier e and surfaces as an ordinary parse error -- the approach based_literal_body already takes for a malformed 8'squark. Valid exponents (1.5e3, 2e-3, 1E6) untouched. Triggers: 1e, 1e+, 99e, 1.5e, parameter real p=2e. BUG 3 -- PREPROCESSOR EOF (preprocessor/parser.rs): previous_range() (self.full_tokens[pos], reached while building the "expected an identifier" diagnostic) and followed_by_bracket_without_space() (self.relevant_tokens[self.pos + 1u32]; "index out of bounds: the len is 2 but the index is 2") indexed the token list directly, out of bounds one past the last token -- a bare "define" ending the file. FIX: both use .get() with a sensible fallback,

Enhancement	Pipeline areas	One line
E-214	hir_lower (expr.rs, stmt.rs), examples/ arraycast_examples/*	whole-array type coercion: a recurring crash class, fixed at its root. An integer Value reaching a float MIR op (feq/fmul/fsub/fdiv) panics mir_opt::const_eval::eval_binary, which has no (Int, Float) case -- "invalid operation fdiv Int(1) Float(..)", exit 101, "please open an issue". (Unfolded, the same defect instead reaches LLVM as i32 = fadd .., ConstantFP:f64 and aborts with "LLVM ERROR: Cannot select" -- one bug, two faces.) FOUR INSTANCES, each patched at its own call site as found: the case discriminant over an integer array (E-33, element type hardcoded real); laplace_/zi_ integer-LITERAL coefficients (b77266ec); laplace_/zi_ integer ARRAY-VARIABLE coefficients (c55812d6 -- the previous fix had reasoned "a whole-array variable reference is already real by its declaration", which is false for an integer array); and an integer case ITEM against a REAL array discriminant (8d0ab057 -- the opcode is chosen from the discriminant, Feq, but the item's elements lower to i32). ROOT CAUSE: inference DOES record the coercion (expect() -> casts.insert(item, Array{Real})), but it lands on the array EXPRESSION, and lower_array_elems_impl decomposes the array and lowers each element itself -- so lower_expr's needs_cast() never saw it and the cast was silently DEAD. Every new array-consuming context therefore re-inherited the trap. FIX: lower_array_elems_impl -- the one place every whole-array consumer passes through -- now honours a recorded cast to a real target (coerce_real = matches!(needs_cast(expr), Some((_, dst)) if *dst.base_type() == Type::Real)), making inference's intent effective for all consumers instead of requiring each call site to ask. Proven in isolation: with the case-site flag disabled the structural guard alone fixes every case repro. lower_case additionally keeps its own discr_op == Opcode::Feq coercion -- choosing the opcode from the discriminant makes matching operand types that function's invariant, independent of inference's bookkeeping. NO MISCOMPILE: the integer spelling of a filter is bit-identical to the '{1.0}' spelling (worst AC diff exactly 0 dB over 13 points) and matches the analytic response to 3e-8 dB; an integer case item still selects its arm. MUTATION-TESTED: reverting the fixes makes all four repros crash (exit 101) again, so the guard is not vacuous. Also folds in two verification guards hardened during the same hunt -- vafautodiff (8->16 checks: cross-derivatives with both arguments live, the blind spot that hid E-185's hypot bug, plus a 44-point

Enhancements not listed (57, 60, 62–64, 69, 72–77, 79–83, 94–95, 98–100) changed no compiler sources — they were validation suites, documentation, benchmark tooling, or ngspice-side work (see the [ngspice report](#)).